

easy-db CLI 命令行工具开发指导书

一、模块概述

1.1 什么是 CLI 命令行工具？

CLI（Command Line Interface，命令行界面） 是一种通过终端/命令行窗口与程序交互的方式。用户输入命令，程序执行并返回结果。

本次任务：为你的 `easy-db` 数据库开发一个 **交互式命令行客户端**，用户可以通过终端输入命令来操作数据库。

1.2 为什么需要 CLI？

用途	说明
手动测试	开发过程中快速测试服务端功能
数据管理	运维人员直接查看和修改数据
演示展示	验收时直观展示功能
调试工具	排查问题时快速执行命令

1.3 效果预览

```
$ java -jar easy-db-cli.jar -h 127.0.0.1 -p 8080
Connecting to localhost:8080...
Connected! Type 'help' for commands.

easy-db> SET name 张三
OK
easy-db> GET name
"张三"
easy-db> SET age 25
OK
easy-db> GET age
"25"
easy-db> KEYS *
1) "name"
2) "age"
easy-db> DEL name
OK
easy-db> GET name
(nil)
easy-db> EXIT
Bye bye!
```

二、功能需求

2.1 连接管理

目标：支持通过命令行参数或交互命令连接服务端。

需求 2.1.1：命令行参数连接

用户启动程序时可以指定服务器地址和端口。

参数	说明	示例
<code>-h</code> 或 <code>--host</code>	服务器地址	<code>-h 127.0.0.1</code>
<code>-p</code> 或 <code>--port</code>	服务器端口	<code>-p 8080</code>
<code>-c</code> 或 <code>--command</code>	直接执行单条命令（非交互模式）	<code>-c "GET name"</code>

启动示例：

```
# 交互模式
java -jar easy-db-cli.jar -h 127.0.0.1 -p 8080

# 单命令模式（执行完自动退出）
java -jar easy-db-cli.jar -h 127.0.0.1 -p 8080 -c "GET name"
```

需求 2.1.2：交互内连接命令

如果启动时未指定连接参数，或需要切换连接，可以在交互内使用 `CONNECT` 命令。

```
easy-db> CONNECT 127.0.0.1 8080
Connecting to 127.0.0.1:8080...
Connected!
easy-db> CONNECT localhost 9090
Switching to localhost:9090...
Connected!
```

验收标准：

- ☐ 支持 `-h` 和 `-p` 参数启动
- ☐ 支持 `CONNECT host port` 命令切换连接
- ☐ 连接成功显示提示信息
- ☐ 连接失败显示错误信息

2.2 核心数据命令

目标：支持服务端已定义的所有命令。

命令	格式	说明	示例
<code>SET</code>	<code>SET key value</code>	存储键值对	<code>SET name 张三</code>
<code>GET</code>	<code>GET key</code>	获取键对应的值	<code>GET name</code>
<code>DEL</code>	<code>DEL key</code>	删除键值对	<code>DEL name</code>
<code>KEYS</code>	<code>KEYS [pattern]</code>	列出所有键，支持通配符	<code>KEYS user:*</code>
<code>EXISTS</code>	<code>EXISTS key</code>	检查键是否存在	<code>EXISTS name</code>
<code>FLUSH</code>	<code>FLUSH</code>	清空所有数据	<code>FLUSH</code>

命令处理要求：

- 命令不区分大小写（`SET` 和 `set` 都识别）
- 参数用空格分隔（支持带空格的 value 用引号包裹，如 `SET name "张 三"`）
- 如果服务端返回错误，CLI 需要显示友好的错误提示

响应格式：

```
# 成功
OK

# 返回值
"张三"

# 空值
(nil)

# 错误
(error) ERR unknown command 'FOO'
```

验收标准：

- ☐ 所有核心命令正常执行
- ☐ 命令不区分大小写
- ☐ 错误信息友好显示

2.3 帮助命令

目标：提供 `HELP` 命令，让用户随时查看可用命令。

```
easy-db> HELP
Available commands:
  SET key value      - 存储键值对
  GET key            - 获取键对应的值
  DEL key            - 删除键值对
  KEYS [pattern]     - 列出所有键
  EXISTS key         - 检查键是否存在
  FLUSH              - 清空所有数据

  CONNECT host port  - 连接服务器
  EXIT / QUIT / Ctrl+C - 退出程序
  HELP               - 显示帮助信息
```

验收标准：

- ☐ `HELP` 命令显示所有可用命令
- ☐ 帮助信息清晰易读

2.4 退出命令

目标：提供多种方式安全退出程序。

方式	说明
<code>EXIT</code> 命令	输入 <code>EXIT</code> 或 <code>QUIT</code> 退出
<code>Ctrl+C</code>	键盘快捷键中断退出
自动重连失败	连续失败 3 次后提示用户选择

退出效果：

```
easy-db> EXIT
Bye bye!
```

验收标准：

- ☐ `EXIT` / `QUIT` 命令正常退出
- ☐ `Ctrl+C` 正常退出
- ☐ 退出前断开与服务端的连接

2.5 增强功能（选做加分）

功能	说明	加分
命令历史	按上下键翻看历史命令（使用 <code>JLine</code> 或 <code>Readline</code> 库）	+5
Tab 补全	输入命令时按 Tab 键自动补全	+3
彩色输出	不同类型输出使用不同颜色（如成功=绿色、错误=红色）	+2
配置文件	支持 <code>~/easy-db-cli.conf</code> 配置文件，设置默认地址	+2
脚本模式	支持 <code>-f script.txt</code> 批量执行命令文件	+3

三、代码设计

3.1 包结构

在 `client` 包下新增 `cli` 子包：

```
src/main/java/client/
├── Client.java           # 已有
├── SocketClient.java     # 已有（供 CLI 调用）
├── cli/                  # ← 新增 CLI 模块
│   ├── CliMain.java     # 入口类，解析命令行参数
│   ├── CliSession.java  # 交互会话管理
│   ├── CommandExecutor.java # 命令执行器
│   ├── CommandParser.java # 命令解析器
│   └── ConsoleReader.java # 控制台输入读取
└── orm/                  # 已有
```

3.2 核心类设计

CliMain.java（程序入口）

```
public class CliMain {
```

```

private static String host = "127.0.0.1";
private static int port = 8080;
private static String singleCommand = null;

public static void main(String[] args) {
    // 1. 解析命令行参数 (-h, -p, -c)
    parseArgs(args);

    // 2. 创建 CliSession
    CliSession session = new CliSession(host, port);

    // 3. 如果是单命令模式, 执行后退出
    if (singleCommand != null) {
        String result = session.execute(singleCommand);
        System.out.println(result);
        session.close();
        return;
    }

    // 4. 否则进入交互模式
    session.startInteractive();
}
}

```

CliSession.java (交互会话管理)

```

public class CliSession {
    private SocketClient client;
    private String host;
    private int port;
    private boolean connected = false;
    private Scanner scanner;

    public CliSession(String host, int port) {
        this.host = host;
        this.port = port;
        this.scanner = new Scanner(System.in);
    }

    public void startInteractive() {
        // 1. 尝试连接服务端
        connect();

        // 2. 打印欢迎信息和提示符
        System.out.println("Connected! Type 'help' for commands.");
        System.out.println();

        // 3. 循环读取用户输入
        while (true) {
            System.out.print("easy-db> ");
            String input = scanner.nextLine().trim();

            if (input.isEmpty()) continue;

            // 处理特殊命令
            if (input.equalsIgnoreCase("EXIT") || input.equalsIgnoreCase("QUIT")) {
                System.out.println("Bye bye!");
                break;
            }
        }
    }
}

```

```

    }
    if (input.equalsIgnoreCase("HELP")) {
        showHelp();
        continue;
    }
    if (input.toUpperCase().startsWith("CONNECT ")) {
        handleConnect(input);
        continue;
    }

    // 执行命令
    String result = execute(input);
    System.out.println(result);
}

// 4. 清理资源
close();
}

public String execute(String command) {
    if (!connected) {
        return "(error) Not connected to server. Use CONNECT host port";
    }
    try {
        return client.sendCommand(command);
    } catch (Exception e) {
        return "(error) " + e.getMessage();
    }
}

private void connect() {
    try {
        // 使用已有的 SocketClient 连接
        client = new SocketClient(host, port);
        client.connect();
        connected = true;
        System.out.println("Connected to " + host + ":" + port);
    } catch (Exception e) {
        System.out.println("(error) Failed to connect: " + e.getMessage());
        connected = false;
    }
}

private void handleConnect(String input) {
    // 解析 "CONNECT host port"
    String[] parts = input.split(" ");
    if (parts.length != 3) {
        System.out.println("(error) Usage: CONNECT host port");
        return;
    }
    this.host = parts[1];
    this.port = Integer.parseInt(parts[2]);
    if (client != null) {
        client.disconnect();
    }
    connect();
}

```

```

private void showHelp() {
    System.out.println("Available commands:");
    System.out.println("  SET key value      - 存储键值对");
    System.out.println("  GET key           - 获取键对应的值");
    System.out.println("  DEL key           - 删除键值对");
    System.out.println("  KEYS [pattern]    - 列出所有键");
    System.out.println("  EXISTS key        - 检查键是否存在");
    System.out.println("  FLUSH             - 清空所有数据");
    System.out.println("");
    System.out.println("  CONNECT host port - 连接服务器");
    System.out.println("  EXIT / QUIT / Ctrl+C - 退出程序");
    System.out.println("  HELP              - 显示帮助信息");
}

public void close() {
    if (client != null) {
        client.disconnect();
    }
    scanner.close();
}
}

```

CommandParser.java (命令解析器, 选做增强)

如果希望支持更复杂的参数 (如带空格的 value) , 可以增加命令解析器:

```

public class CommandParser {
    /**
     * 解析命令字符串
     * 支持用引号包裹包含空格的参数
     * 示例: SET name "张 三"
     * 返回: ["SET", "name", "张 三"]
     */
    public static List<String> parse(String input) {
        List<String> args = new ArrayList<>();
        StringBuilder current = new StringBuilder();
        boolean inQuotes = false;

        for (char c : input.toCharArray()) {
            if (c == '"') {
                inQuotes = !inQuotes;
            } else if (c == ' ' && !inQuotes) {
                if (current.length() > 0) {
                    args.add(current.toString());
                    current = new StringBuilder();
                }
            } else {
                current.append(c);
            }
        }
        if (current.length() > 0) {
            args.add(current.toString());
        }
        return args;
    }
}

```

四、使用示例

4.1 交互模式完整演示

```
# 启动 CLI 工具 (连接到 localhost:8080)
$ java -jar easy-db-cli.jar -h 127.0.0.1 -p 8080
Connecting to 127.0.0.1:8080...
Connected! Type 'help' for commands.

# 存储数据
easy-db> SET user:001 张三
OK

easy-db> SET user:002 李四
OK

easy-db> SET user:003 王五
OK

# 查询数据
easy-db> GET user:001
"张三"

# 列出所有键
easy-db> KEYS *
1) "user:001"
2) "user:002"
3) "user:003"

# 列出匹配的键
easy-db> KEYS user:00*
1) "user:001"
2) "user:002"
3) "user:003"

# 检查存在
easy-db> EXISTS user:001
true

easy-db> EXISTS user:999
false

# 删除数据
easy-db> DEL user:002
OK

easy-db> GET user:002
(nil)

# 查看帮助
easy-db> HELP
Available commands:
  SET key value      - 存储键值对
  GET key            - 获取键对应的值
  DEL key            - 删除键值对
  KEYS [pattern]     - 列出所有键
  EXISTS key         - 检查键是否存在
```



```
FLUSH                - 清空所有数据

CONNECT host port    - 连接服务器
EXIT / QUIT / Ctrl+C - 退出程序
HELP                 - 显示帮助信息
```

切换连接

```
easy-db> CONNECT localhost 9090
Switching to localhost:9090...
Connected!
```

清空数据

```
easy-db> FLUSH
OK
```

退出

```
easy-db> EXIT
Bye bye!
```

4.2 单命令模式

直接执行单条命令（适合脚本调用）

```
$ java -jar easy-db-cli.jar -h 127.0.0.1 -p 8080 -c "GET user:001"
"张三"
```

执行多条命令（多条用 ; 分隔）

```
$ java -jar easy-db-cli.jar -h 127.0.0.1 -p 8080 -c "SET a 1; SET b 2; KEYS *"
OK
OK
1) "a"
2) "b"
```

4.3 脚本批量执行（选做）

准备脚本文件 script.txt

```
$ cat script.txt
SET user:001 张三
SET user:002 李四
KEYS *
GET user:001
```

执行脚本

```
$ java -jar easy-db-cli.jar -h 127.0.0.1 -p 8080 -f script.txt
OK
OK
1) "user:001"
2) "user:002"
"张三"
```

五、开发任务清单

阶段	任务	状态
阶段一	创建 <code>cli</code> 包和 <code>CliMain</code> 类	☐
阶段二	实现命令行参数解析 (-h, -p, -c)	☐
阶段三	实现 <code>CliSession</code> 交互循环	☐
阶段四	实现命令执行（调用 <code>SocketClient</code> ）	☐
阶段五	实现 <code>HELP</code> 、 <code>CONNECT</code> 、 <code>EXIT</code> 命令	☐
阶段六	错误处理和友好提示	☐
阶段七（选做）	命令历史（ <code>JLine</code> ）	☐
阶段八（选做）	脚本批量执行	☐

六、常见问题

Q1: `Scanner.nextLine()` 读取中文乱码？

A: 确保终端编码为 UTF-8，或在启动时指定编码：

```
java -Dfile.encoding=UTF-8 -jar easy-db-cli.jar
```

Q2: 如何实现命令历史（上下键）？

A: 推荐使用 `JLine` 库：

```
<dependency>
  <groupId>org.jline</groupId>
  <artifactId>jline</artifactId>
  <version>3.23.0</version>
</dependency>
```

使用 `LineReader` 替代 `Scanner` 即可获得历史功能。

Q3: 如何正确处理 `Ctrl+C`？

A: 添加关闭钩子（Shutdown Hook）：

```
Runtime.getRuntime().addShutdownHook(new Thread(() -> {
    System.out.println("\nBye bye!");
    // 清理资源
}));
```

七、提交要求

1. 代码：CLI 模块全部源码，Git 提交记录
2. 打包：提供可执行的 `easy-db-cli.jar`
3. 报告：在课程设计报告中增加 **CLI 命令行工具** 章节，内容包括：
 - 设计思路
 - 核心类说明

- 使用示例截图（至少 5 张）
- 遇到的困难与解决

4. 演示：验收时现场演示 CLI 工具的使用

祝同学们顺利完成 CLI 工具开发！